Python 08: regular expressions

https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM18.html

Quick links:
Click on any ⌂ to get back here!

What are regular expressions!?

A regular expression is a sequence of characters that define a search pattern.

Usually such patterns are used by string-searching algorithms for "find" or "find and replace" operations on strings, or for input validation.

- Regular expressions (aka. "regex") are a special mini-language for describing patterns in strings.
- Handy for working with patterns in DNA/protein, also text files of many different types.
- Also find them in other tools, e.g. text editors, and the `grep` command we already know how to use in a shell

---

Regular expression module

```
# How did we find strings before?
dna = 'ATCGCGAATTCAC'
dna.find('GCGA')
```

```
3
```

```
dna.find('G')
```

```
3
```

The tools for using regular expressions live in the `re` module.

We have to import the module, then use the module name when running functions.

```
# Do a search for the letter 'a' in the string 'abc'
search('a', 'abc')
```

```
Traceback (most recent call last):
  File '<stdin>', line 1, in <module>
NameError: name 'search' is not defined
```

```
# Import the regular expression module re so that we can do pattern matching
import re
```

```
# Try again
search('a', 'abc')
```

```
Traceback (most recent call last):
  File '<stdin>', line 1, in <module>
NameError: name 'search' is not defined
```

```
# Try again, using the module name to indicate where the search tool is
```

```
re.search('a', 'abc')
```

```
<_sre.SRE_Match object; span=(0, 1), match='a'>
```

```
# Try again, printing it
```

```
print(re.search('a', 'abc'))
```

```
<_sre.SRE_Match object; span=(0, 1), match='a'>
```

```
# re.search() acts more like a True / False
```

```
if re.search('a', 'abc') :
  print('Found the a')
else :
  print('Failed to find a')
```

```
Found the a
```

```
if re.search('z', 'abc') :
  print('Found the z')
else :
  print('Failed!')
```

```
Failed!
```

```
# We think it should be "False", because there is no z in abc?
```

```
re.search('z', 'abc')
```

```

```

```
# We think we are happy with this response...
```

```
re.search('z', 'abc') == True
```

```
False
```

```
# But actually, it ISN'T a True/False response
```

```
re.search('z', 'abc') == False # We might think that this should be "True"
```

```
False
```

```
type(re.search('z', 'abc'))
```

```
<class 'NoneType'>
```

```
# Quick help: DOT TAB TAB
```

```
re.
```

```
re.A            re.I            re.M            re.Pattern()    re.T            re.VERBOSE
re.ASCII        re.IGNORECASE   re.MULTILINE    re.RegexFlag(   re.TEMPLATE     re.X
re.DEBUG        re.L            re.Match()      re.S            re.U            re.compile(
re.DOTALL       re.LOCALE       re.NOFLAG       re.Scanner(     re.UNICODE      re.copyreg
```

More help

help(re)

```
Help on package re:

NAME
    re - Support for regular expressions (RE).

MODULE REFERENCE
    https://docs.python.org/3.12/library/re.html

    The following documentation is automatically generated from the Python
    source files.  It may be incomplete, incorrect or include features that
    are considered implementation detail and may vary between Python
    implementations.  When in doubt, consult the module reference at the
    location listed above.

DESCRIPTION
    This module provides regular expression matching operations similar to
    those found in Perl.  It supports both 8-bit and Unicode strings; both
    the pattern and the strings being processed can contain null bytes and
    characters outside the US ASCII range.

    Regular expressions can contain both special and ordinary characters.
    Most ordinary characters, like "A", "a", or "0", are the simplest
    regular expressions; they simply match themselves.  You can
    concatenate ordinary characters, so last matches the string 'last'.

    The special characters are:
        "."      Matches any character except a newline.
        "^"      Matches the start of the string.
        "$"      Matches the end of the string or just before the newline at
                 the end of the string.
        "*"      Matches 0 or more (greedy) repetitions of the preceding RE.
                 Greedy means that it will match as many repetitions as possible.
        "+"      Matches 1 or more (greedy) repetitions of the preceding RE.
        "?"      Matches 0 or 1 (greedy) of the preceding RE.
        *?,+?,?? Non-greedy versions of the previous three special characters.
        {m,n}    Matches from m to n repetitions of the preceding RE.
        {m,n}?   Non-greedy version of the above.
        "\\"     Either escapes special characters or signals a special sequence.
        []       Indicates a set of characters.
                 A "^" as the first character indicates a complementing set.
        "|"      A|B, creates an RE that will match either A or B.
        (...)    Matches the RE inside the parentheses.
                 The contents can be retrieved or matched later in the string.
        (?aiLmsux) The letters set the corresponding flags defined below.
        (?:...)  Non-grouping version of regular parentheses.
....
```

Raw strings

Prefix a string with `r` to turn it into a 'raw string'.

Raw strings are good if you need to ignore special characters when doing a search. Python's raw strings are just a way to tell the Python interpreter that it should interpret backslashes as literal slashes.

```
# What we are used to...
my_string = 'hello\nworld\t!'
print(my_string)
```

```
hello
world   !
```

```
# Doing it as a raw string...
my_raw_string = r'hello\nworld\t!'
print(my_raw_string)
```

```
hello\nworld\t!
```

```
# Other example: one backslash
myvar= '\n'
type(myvar)
```

```
<class 'str'>
```

```
print(myvar)

myvar
```

```
'\n'
```

```
# Other example: raw, one backslash
myvar= r'\n'
type(myvar)
```

```
<class 'str'>
```

```
print(myvar)
\n
myvar
'\\n'
```

```
# Other example: two backslashes
myvar= '\\n'
type(myvar)
<class 'str'>
print(myvar)
\n
myvar
'\\n'
```

```
# Other example: raw, two backslashes
myvar= r'\\n'
type(myvar)
<class 'str'>
print(myvar)
\\n
myvar
'\\\\n'
```

Searching for patterns

As we have seen, `re.search()` takes two arguments:

1. a pattern to look for
2. the string to search in for the pattern

`re.search(pattern,string)`

It searches for the pattern in the string and returns either **True** or **None**.

 # Searching for a simple pattern in a string

```
dna = 'ATCGCGAATTCAC'
if re.search(r'GAATTC', dna):
    print('Restriction site GAATTC found.')
else:
    print('Restriction site GAATTC not found.')
```

Restriction site GAATTC found.

Note that we don't really need a raw string in the example above, but it's a good habit to get into.

Alternatives

Sometimes, there might be two (or more) different possibilities that we want to search for, e.g. A or T at a particular base position

When this is the case, we surround the possibilities with ordinary brackets `( )` and separate the possibilities with pipe characters `|`.

 # Searching for a simple pattern G-AorT-CG in a string

```
dna = 'ATCGCGAATTCAC'
if re.search(r'G(A|T)CG', dna):
    print('Restriction site G-AorT-CG found.')
else:
    print('Restriction site G-AorT-CG not found.')
```

Restriction site G-AorT-CG not found.

Character groups

A very common type of search situation is when we want to allow any one of a list of characters.

```
# Any of four bases at one position, here's one way of doing it
# Looking for GC-then any base-GC
dna = 'ATCGCGAATTCAC'
if re.search(r'GC(A|T|G|C)GC', dna):
    print('Found what we were looking for...')
```

No output because it wasnt found...

```
# Here's another way of doing it
if re.search(r'GC[GATC]GC', dna):
    print('Found what we were looking for...')
```

No output because it wasnt found...

```
# Different search, found one this time!
if re.search(r'CG[GATC]AT', dna):
    print('Found what we were looking for...')
```

```
Found what we were looking for...
```

Sometimes it's easier to describe a character group by listing the characters that are **not** allowed, e.g. ambiguous bases in DNA sequence are a good example of such a situation.

Special rule for when using re: if the character group starts with `^` then it means any character `except` the ones in the "negated character group".

Remember the DNA sequence ambiguity codes....?!

Ambiguity code	DNA bases
N	a**N**y base
R	A or G (pu**R**ine)
Y	C or T (p**Y**rimidine)
S	G or C
W	A or T
K	G or T
M	A or C
B	C or G or T
D	A or G or T
H	A or C or T
V	A or C or G

```
# A sequence with an ambiguous base in it
dna = 'ATCGCGYAATTCAC'
```

```
# Search to see if there are bases that are NOT G or A or T or C
if re.search(r'[^GATC]', dna):
    print('Ambiguous base found!')
else :
    print('Sequence has no ambiguous bases in it.')
```

```
Ambiguous base found!
```

```
# Search a subset of the sequence to see if there are any ambiguous bases
dna = 'ATCGCGYAATTCAC'
if re.search(r'[^GATC]', dna[0:6]):
    print('Ambiguous base found!')
else :
    print('Sequence has no ambiguous bases in it.')
```

```
Sequence has no ambiguous bases in it.
```

```
# Don't forget the quick help: DOT TAB TAB
```

```
re.
```

```
re.A            re.I            re.M            re.Pattern()    re.T            re.VERBOSE
re.ASCII        re.IGNORECASE   re.MULTILINE    re.RegexFlag(   re.TEMPLATE     re.X
re.DEBUG        re.L            re.Match()      re.S            re.U            re.compile(
re.DOTALL       re.LOCALE       re.NOFLAG       re.Scanner(     re.UNICODE      re.copyreg
```

Match 'shortcuts': the 'cheat sheet'

 # Non-special chars match themselves. Exceptions are special characters

What	What it means
\	Escape special character or start a sequence.
.	Match any character except newline
^	Match start of the string
$	Match end of the string
[]	Enclose a set of matchable characters
()	Create capture group, & indicate precedence

 # After a '[', used to 'enclose a set', the only special chars are:

What	What it means
]	End the set, if not the 1st char
-	A range, eg. a-c matches a, b or c
^	If it is the 1st character, **negate** the entire set (i.e. 'none of these')

 # Quantifiers: how many?

What	What it means
{m}	Exactly m repetitions
{m,n}	From m (default 0) to n (default infinity) repetitions

*	0 or more. Same as {,}
+	1 or more. Same as {1,}
?	0 or 1. Same as {,1}

 # Special string sequences

What	What it means
\A	Start of string
\b	Match empty string at word (\w+) boundary
\B	Match empty string not at word boundary
\d	Digit
\D	Non-digit
\s	Whitespace [\t\n\r\f\v]
\S	Non-whitespace
\w	Alphanumeric: [0-9a-zA-Z_]
\W	Non-alphanumeric
\Z	End of string

 # Special character escapes are much like those already escaped in Python string literals. Hence regex '\n' is same as regex '\\n'

What	What it means
\a	ASCII Bell (BEL)
\f	ASCII Formfeed
\n	ASCII Linefeed
\r	ASCII Carriage return
\t	ASCII Tab
\v	ASCII Vertical tab

\\ A single backslash

```
# We know from the above tables that a full stop (also known as a 'dot', or 'period') stands for any character.
dna = 'ATCGCGYAATTCAC'
```

```
# Let's look for 'T something A'
re.search(r'T.A', dna)
```

```
<_sre.SRE_Match object; span=(10, 13), match='TCA'>
```

```
re.search(r'T.A', dna).span()
```

```
(10, 13)
```

```
# Let's look for 'T something A' from index position 11 onwards
```

```
re.search(r'T.A', dna[11:])
```

```

```

```
# Remember, re.search() essentially behaves as a boolean True/False
# (but is actually a True/None)
```

```
if re.search(r'T.A', dna):
    print('\'T something A\' found!')
else :
    print('\'T something A\' not present in the sequence.')
```

```
'T something A' found!
```

Quantifiers

Another type of variation we might encounter, especially in DNA sequences, is the number of times something (e.g. a base) is repeated.

A question mark `?` means the thing preceding it is optional.

- In the pattern `GCA?Y` the A is optional.
- The pattern will match GCAY and GCY.

A plus `+` means that the thing preceding it can be repeated more than once.

- In the pattern `GCA+Y` the A can be repeated
- it matches GCAY, GCAAY, GCAAAY, etc. but not GCY

An asterisk `*` is the most flexible quantifier: the thing preceding it is optional, but can also be repeated.

- The pattern `GCA*Y` will match GCY, GCAY, GCAAY, GCAAAY, etc.

For more specificity, we can specify a minimum and maximum number of repetitions.

- `GCA{2,4}Y` allows 2-4 As, so will match GCAAY, GCAAAY and GCAAAAY but not GCY or GCAY or GCAAAAAY, etc.

Positions

Unlike all the options above, positions specify **where** the pattern has to match the string. These are just like the ones we used in our `awk` scripting with `grep` at the beginning of the course.

`^` means the start, so the pattern `^G` will match GATC but not AGATC.

Note: this is NOT the same as when the `^` is at the beginning of a square brackets term: (e.g. [^GATC] as described earlier, which means '**not** G or A or T or C').

`$` means the end, so the pattern `C$` will match GATC but not GATCA.

 # Starts with A?

```
if re.search(r'^A', 'AGTC'):
    print('Starts with A')
else :
    print('Does not start with A')
```

Starts with A

 # Ends with A?

```
if re.search(r'A$', 'AGTC'):
    print('Ends with A')
else :
    print('Doesn\'t end with A')
```

Doesn't end with A

 # Start with A or G?

```
if re.search(r'^[AG]', 'TGTC'):
    print('Starts with A or G')
else :
    print('Doesn\'t start with A or G')
```

Doesn't start with A or G

 # Starts with A or G AND ends with C?

```
if re.search(r'^[AG]', 'TGTC') and re.search(r'C$', 'TGTC'):
    print('Starts with A or G AND ends with C')
else :
    print('One or more of the conditions not met...')
```

One or more of the conditions not met...

 # Starts with A or G OR ends with C?

```
if re.search(r'^[AG]', 'TGTC') or re.search(r'C$', 'TGTC'):
    print('Starts with A or G, OR ends with a C')
else :
    print('Neither starts with A or G, NOR ends with a C')
```

Starts with A or G, OR ends with a C

 # Starts with A or G OR ends with T?

```
if re.search(r'^[AG]', 'TGTC') or re.search(r'T$', 'TGTC'):
    print('One condition met')
else :
    print('Neither condition met')
```

Neither condition met

How useful are those last two print statements?!

- good: they can help tell me that the `if` statement is working as I want it to
- good: I can easily test it using differing things to search with or search in
- good: I can comment them out later if/when I dont need them any more
- good: I could be sending the output to a write connection to a file
- bad: they are not very meaningful to anyone other than me really!

Combinations

The real power of regular expressions comes from combining most/all of these features.

Here's a description of an example full-length messenger RNA:

1. an initial G,
2. followed by a 5' UTR region of 10-150 bases,
3. then an ATG methionine start codon,
4. coding region of 30 to 1000 bases,
5. then a TGA stop codon (how would you include the other stop codons, TAA and TAG?!)
6. then a 3' UTR of 20-50 bases,
7. and then, finally, followed by a poly(A) tail of 5-10 As.

Here's a regular expression that describes that messenger RNA:

```
^G[GATC]{10,150}ATG[GATC]{30,1000}TGA[GATC]{20,50}A{5,10}$
```

Look at the features:

1. string must start with `G`
2. then between 10 and 150 bases that can only be A/T/G/C `[GATC]{10,150}`
3. then an `ATG`
4. then between 30 and 1000 bases that can only be A/T/G/C `[GATC]{30,1000}`
5. then a `TGA`
6. then between 20 and 50 bases that can only be A/T/G/C `[GATC]{20,50}`

7. string must end with between 5 and 10 consecutive As `A{5,10}`

Other stuff: what was the match?

As we saw at the start, `re.search()` behaves a *bit* like a True / False evaluation, but in fact it returns a **re match** object.

Luckily there are methods to get at what the object contains.

```
# Our non-GATC ambiguous base example again
```

```
dna = 'ATCGCGYAATTCAC'
if re.search(r'[^GATC]', dna):
    print('Ambiguous base found!')
else :
    print('Sequence has no ambiguous bases in it.')
```

```
Ambiguous base found!
```

```
# OK, so we have found a non-GATC base, but what was it?
dna = 'ATCGCGYAATTCAC'
match_obj = re.search(r'[^GATC]', dna)
```

```
# match_obj is now our match object
match_obj
<_sre.SRE_Match object; span=(6, 7), match='Y'>
type(match_obj)
<class '_sre.SRE_Match'>
```

```
# Using the True/False behaviour
# Calling group() on the match object will tell us which
# portion of our input string matched the pattern
```

```
if match_obj :    # if it was "True"
    print('Ambiguous base found!')
    ambig = match_obj.group()
    print('The base is ' + ambig)
```

```
Ambiguous base found!
The base is Y
```

```
# Do a different search, this time for A or T
```

```
match_obj2 = re.search(r'[AT]', dna)
```

```
match_obj2
```

```
<_sre.SRE_Match object; span=(0, 1), match='A'>
```

```
match_obj2.group()
```

```
'A'
```

```
# But surely there's more than one A or T bases?!
```

```
dna.upper().count('A') + dna.upper().count('T')
```

```
7
```

```
# Help on our match_obj DOT TAB TAB
```

```
match_obj.
```

```
match_obj.end(          match_obj.groupdict(  match_obj.pos          match_obj.start(
match_obj.endpos        match_obj.groups(     match_obj.re           match_obj.string
match_obj.expand(       match_obj.lastgroup   match_obj.regs
match_obj.group(        match_obj.lastindex   match_obj.span(
```

Extracting multiple groups

What if there are more than two different things we want to find at the same time, i.e. more than one bit of the pattern we are interested in?

In terms of a regex search term, this can be expressed as
`.+ .+`
which means 'one or more characters, followed by a space, followed by one or more characters'.

However, if we want to keep the two 'bits' separate for different uses later on, we need to **capture** the parts of the pattern that we want to store, and we do this using ordinary brackets:
`(.+) (.+)`

```
# Parsing genus and species from a list of animals
lots_of_animals = [
  'Homo sapiens',
  'Mus musculus',
  'Erinaceus europaeus',
  'Columba livia',
  'Sciurus vulgaris',
  'Gorilla'
    ]
# for loop to consider each list element at a time
for this_animal in lots_of_animals :
   new_match_obj = re.search('(.+) (.+)',this_animal)
   print(this_animal+': genus is ' + new_match_obj.group(1)+', species is ' + new_match_ob
```

```
Homo sapiens: genus is Homo, species is sapiens
Mus musculus: genus is Mus, species is musculus
Erinaceus europaeus: genus is Erinaceus, species is europaeus
Columba livia: genus is Columba, species is livia
Sciurus vulgaris: genus is Sciurus, species is vulgaris
Traceback (most recent call last):
  File '<stdin>', line 3, in <module>
AttributeError: 'NoneType' object has no attribute 'group'
```

```
# We got an error, so what does the last match object contain? Seemingly nothing!
new_match_obj
```

```
# Using DOT TAB TAB we get something quite different this time
```

```
new_match_obj.
```

```
new_match_obj.__
```

```
# Using DOT TAB TAB again
```

```
nnew_match_obj.__bool__(          new_match_obj.__getattribute__(   new_match_obj.__new__
new_match_obj.__class__(          new_match_obj.__gt__(             new_match_obj.__reduce
new_match_obj.__delattr__(        new_match_obj.__hash__(           new_match_obj.__reduce
new_match_obj.__dir__(            new_match_obj.__init__(           new_match_obj.__repr__
new_match_obj.__doc__             new_match_obj.__init_subclass__(  new_match_obj.__setatt
new_match_obj.__eq__(             new_match_obj.__le__(             new_match_obj.__sizeof
new_match_obj.__format__(         new_match_obj.__lt__(             new_match_obj.__str__(
new_match_obj.__ge__(             new_match_obj.__ne__(             new_match_obj.__subcla
```

```
type(new_match_obj)
```

```
<class 'NoneType'>
```

```
# Remember that the match object acts a bit like a True / False,
```

```
# but actually, it is a True / None situation
```

```
# Let's put in an error trap that "allows" failure
```

```
for this_animal in lots_of_animals :
   new_match_obj = re.search('(.+) (.+)',this_animal)
   if new_match_obj :
     print(this_animal + ': genus is ' + new_match_obj.group(1) + ', species is ' + new_ma
   else :
     print('Unable to find a full genus and species pair for ' + this_animal)
```

```
Homo sapiens: genus is Homo, species is sapiens
Mus musculus: genus is Mus, species is musculus
Erinaceus europaeus: genus is Erinaceus, species is europaeus
Columba livia: genus is Columba, species is livia
Sciurus vulgaris: genus is Sciurus, species is vulgaris
Unable to find a full genus and species pair for Gorilla
```

```
# Why did we use group(1) and group(2), not 0 and 1 ?!
```

```
dna
```

```
'ATCGCGYAATTCAC'
```

```
re.search('(.+)Y(.+)',dna)
```

```
<_sre.SRE_Match object; span=(0, 14), match='ATCGCGYAATTCAC'>
```

```
re.search('(.+)Y(.+)',dna).group(0)
```

```
'ATCGCGYAATTCAC'
```

```
re.search('(.+)Y(.+)',dna).group(1)
```

```
'ATCGCG'
```

```
re.search('(.+)Y(.+)',dna).group(2)
```

```
'AATTCAC'
```

```
# We could do thing slightly differently, and capture all
re.search('(.+)(Y)(.+)',dna).group(0)
```

```
'ATCGCGYAATTCAC'
```

```
re.search('(.+)(Y)(.+)',dna).group(1)
```

```
'ATCGCG'
```

```
re.search('(.+)(Y)(.+)',dna).group(2)
```

```
'Y'
```

```
re.search('(.+)(Y)(.+)',dna).group(3)
```

```
'AATTCAC'
```

```
# We can also use a string variable to form our search term
look_for_me='AA'
re.search('(.+)' + look_for_me + '(.+)',dna)
```

```
<_sre.SRE_Match object; span=(0, 14), match='ATCGCGYAATTCAC'>
```

```
newsearch = re.search('(.+)' + look_for_me + '(.+)',dna)
print(newsearch.group(0), 'was searched with', look_for_me ,'and we found\n\t1. ', newsearch.group(1) ,'\n\t ', look_for_me , '\n\t2. ' ,newsearch.group(2))
```

```
ATCGCGYAATTCAC was searched with AA and we found
        1.  ATCGCGY
            AA
        2.  TTCAC
```

```
# Still can't mix int and str: try it with an integer!
look_for_me=3
newsearch = re.search('(.+)' + look_for_me + '(.+)',dna)
```

```
Traceback (most recent call last):
  File '<stdin>', line 1, in <module>
TypeError: must be str, not int
```

Getting the position

Another thing we can do is get the position of the match with `start()` and `end()`

```
dna = 'ATCGCGYKNAATTCAC'
match_obj = re.search(r'[^GATC]', dna)
```

```
# match_obj is now a match object
match_obj
```

```
<_sre.SRE_Match object; span=(6, 7), match='Y'>
```

```
if match_obj:
    print('An ambiguous base found!')
    print('The base is ' + match_obj.group())
    print('The base is at Python position ' + str(match_obj.start()))
```

```
An ambiguous base found!
The base is Y
The base is at Python position 6
```

```
# Errr... what about the K and N bases!?
```

Patience, please... we will do that shortly with `re.findall()`

Splitting a string with a regex

`re.split()` works just like regular `split()`, but takes a regular expression pattern as the separator.

`re.split()` excludes the portion of the string that matched the pattern and just keeps the non-matching portion(s).

```
# Here's a fairly ambiguous bit of sequence
dna = 'ACTNGCATRGCYTACYGTYACGATSCGAWTCG'
```

```
# Let's split the sequence whenever we see a Y base, we get a list of strings
re.split(r'Y', dna)
```

```
['ACTNGCATRGC', 'TAC', 'GT', 'ACGATSCGAWTCG']
```

```
# Let's split whenever there is an A base
re.split(r'A', dna)
```

```
['', 'CTNGC', 'TRGCYT', 'CYGTY', 'CG', 'TSCG', 'WTCG']
```

```
# Let's split whenever there is a YA pair
re.split(r'YA', dna)
```

```
['ACTNGCATRGCYTACYGT', 'CGATSCGAWTCG']
```

```
# Let's split whenever there is a AY pair
re.split(r'AY', dna)
```

```
['ACTNGCATRGCYTACYGTYACGATSCGAWTCG']
```

```
# Let's split whenever we see either A or Y
re.split(r'[AY]', dna)
```

```
['', 'CTNGC', 'TRGC', 'T', 'C', 'GT', '', 'CG', 'TSCG', 'WTCG']
```

```
# Let's split the sequence whenever we see a non-GATC base
re.split(r'[^GATC]', dna)
['ACT', 'GCAT', 'GC', 'TAC', 'GT', 'ACGAT', 'CGA', 'TCG']
```

```
# Let's split the sequence whenever we see a GATC base, we get a list again
re.split(r'[GATC]', dna)
['', '', '', 'N', '', '', '', 'R', '', 'Y', '', '', 'Y', '', 'Y', '', '', '', '', 'S', '', '', 'W', '', '', '']
```

```
# What is the non-redundant set of 'post-split' sequences in the list? Yours may be different...
list(set(re.split(r'[GATC]', dna)))
['', 'W', 'Y', 'N', 'S', 'R']
# Sorted, please!
list(set(re.split(r'[GATC]', dna))).sort()

```

```
# Huh? Do I need to print it?
print(list(set(re.split(r'[GATC]', dna))).sort())
None
type(print(list(set(re.split(r'[GATC]', dna))).sort()))
None
<class 'NoneType'>
```

```
# Try again....
NRSet=list(set(re.split(r'[GATC]', dna)))
NRSet
['', 'W', 'Y', 'N', 'S', 'R']
NRSet.sort()
NRSet
['', 'N', 'R', 'S', 'W', 'Y']
```

```
NRSet[0]
''
len(NRSet)
6
```

```
len(NRSet[0])
```

```
0
```

```
len(NRSet[1])
```

```
1
```

```
len(NRSet[2])
```

```
1
```

```
# If we wanted to remove the '' could we not just look for it?
```

```
re.search(r'',NRSet)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/lib/python3.12/re/__init__.py", line 177, in search
    return _compile(pattern, flags).search(string)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: expected string or bytes-like object, got 'list'
```

```
# Is it the quotes?
```

```
re.search(r"''",NRSet)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/lib/python3.12/re/__init__.py", line 177, in search
    return _compile(pattern, flags).search(string)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: expected string or bytes-like object, got 'list'
```

```
# Element by element approach, just keeping the ones we want?
not_empty=[]
for e in NRSet :
   if len(e) > 0 :
      not_empty.append(e)
```

```
not_empty
```

```
['N', 'R', 'S', 'W', 'Y']
```

```
len(not_empty)
```

```
5
```

```
# Quick check
```

```
'' in NRSet
```

```
True
```

```
'' in not_empty
```

```
False
```

```
# A bit prettier (but unsorted)?
```

```
print(', '.join(list(set(re.split(r'[GATC]', dna)))))
```

```
, Y, R, S, W, N
```

```
# We could even make the output into a string...
'|'.join(list(set(re.split(r'[GATC]', dna))))
```

```
'|W|Y|N|S|R'
```

```
# We could even make the output into a contiguous string...
''.join(list(set(re.split(r'[GATC]', dna))))
```

```
'WYNSR'
```

```
# Using the sorted version...
''.join(NRSet)
```

```
'NRSWY'
```

```
# We could turn it back into a list if we wanted to!
list(''.join(NRSet))
```

```
['N', 'R', 'S', 'W', 'Y']
```

Alternative method, if we know which one to `pop()`

```
NRSet
```

```
['', 'N', 'R', 'S', 'W', 'Y']
```

```
NRSet.pop(0)
```

```
''
```

```
NRSet
```

```
['N', 'R', 'S', 'W', 'Y']
```

Multiple finds

`re.findall()` can be used, for example, to find multiple instances of found characters.

```
# Here's what we did earlier
```

```
dna = 'ATCGCGYKNAATTCAC'
re.search(r'[^GATC]', dna)
```

```
<_sre.SRE_Match object; span=(6, 7), match='Y'>
```

```
# Here's what we get using re.findall()
re.findall('[^GATC]',dna)
```

```
['Y', 'K', 'N']
```

```
# Here's what we get using re.findall()
# and a slightly different search term
re.findall('[^GAT]',dna)
```

```
['C', 'C', 'Y', 'K', 'N', 'C', 'C']
```

```
set(re.findall('[^GAT]',dna))
```

```
{'K', 'C', 'Y', 'N'}
```

```
# Here's another DNA sequence
dna = 'ACTGCATTATATCGTACGAAATTATACGCGCG'
```

```
# Let's find all A bases
re.findall(r'A', dna)
# Output is a list
```

```
['A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A']
```

```
dna.count('A')
```

```
10
```

```
len(re.findall(r'A', dna))
```

```
10
```

```
# Let's find all runs of A/T that are at least four bases long
```

```
re.findall(r'[AT]{4,}', dna)
```

```
['ATTATAT', 'AAATTATA']
```

```
# Let's find all runs of A/T that are at least two bases long
```

```
re.findall(r'[AT]{2,}', dna)
```

```
# The output doesn't include overlapping stretches as separate 'finds' (see below!)
```

```
['ATTATAT', 'TA', 'AAATTATA']
```

Finding and processing multiple matches

Some problems require complete match objects for multiple matches, for these we need to use `re.finditer()`

```
# Here's yet another DNA sequence
```

```
dna = 'ACTGCATTATATCGTACGAAATTATACGCGCG'
```

```
re.finditer(r'A', dna)
```

```
<callable_iterator object at 0x7fe8cd866b38>
```

```
# How many finds?
```

```
len(re.finditer(r'A', dna))
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: object of type 'callable_iterator' has no len()
```

```
list(re.finditer(r'A', dna))
```

```
[<_sre.SRE_Match object; span=(0, 1), match='A'>,
 <_sre.SRE_Match object; span=(5, 6), match='A'>,
 <_sre.SRE_Match object; span=(8, 9), match='A'>,
 <_sre.SRE_Match object; span=(10, 11), match='A'>,
 <_sre.SRE_Match object; span=(15, 16), match='A'>,
 <_sre.SRE_Match object; span=(18, 19), match='A'>,
 <_sre.SRE_Match object; span=(19, 20), match='A'>,
 <_sre.SRE_Match object; span=(20, 21), match='A'>,
 <_sre.SRE_Match object; span=(23, 24), match='A'>]
```

```
len(list(re.finditer(r'A', dna)))
```

```
10
```

```
# What are the start and stop positions of all stretches of A or T at least 4 bases long?
```

```
AorT_runs = re.finditer(r'[AT]{4,}', dna)
```

```
# What were the stretches?
```

```
AorT_runs
```

```
<callable-iterator object at 0x7f17d79f6710>
```

```
type(AorT_runs)
```

```
<class 'callable_iterator'>
```

```
list(AorT_runs)
```

```
[<re.Match object; span=(5, 12), match='ATTATAT'>, <re.Match object; span=(18, 26), match=
```

```
# Output where the "A or T" runs are in the sequence and print them out
```

```
counter=0
for ATs in AorT_runs:
    counter += 1  # same as counter = counter + 1
    run_start = ATs.start()
    run_end = ATs.end()
    print('A/T run: ' +str(counter)+' extends from ' +str(run_start) \
    + ' to ' +str(run_end)+' and has sequence ' +str(dna[run_start:run_end]))
```

```
A/T run: 1 extends from 5 to 12 and has sequence ATTATAT
A/T run: 2 extends from 18 to 26 and has sequence AAATTATA
```

```
# What are the start/stop positions of all stretches of A or T 3 bases long?
AorT_runs_v2 = re.finditer(r'[AT]{3,3}', dna)
```

```
# Output where the "A or T" triplets are in the sequence and print them out
counter=0
for ATs in AorT_runs_v2:
    counter += 1
    run_start = ATs.start()
    run_end = ATs.end()
    print('A/T triplet: ' +str(counter)+' extends from ' +str(run_start) \
    + ' to ' +str(run_end)+' and has sequence ' +str(dna[run_start:run_end]))
```

```
A/T triplet: 1 extends from 5 to 8 and has sequence ATT
A/T triplet: 2 extends from 8 to 11 and has sequence ATA
A/T triplet: 3 extends from 18 to 21 and has sequence AAA
A/T triplet: 4 extends from 21 to 24 and has sequence TTA
```

Lookahead assertions

What if we want to know all the start/stop positions of all stretches of AorT 3 bases long, this time **allowing overlap**?

`(?=xyz)` is True if `xyz` matches, but doesn't consume any of the string: this is called a lookahead assertion.

```
# Let's try it in our search for A or T triplets
stringlength=3
AorT_runs_v3 = re.finditer(r'(?= [AT]{3,3})', dna)
```

```
# Output where any overlapping A or T triplets are in the sequence and print them out
counter=0
for ATs in AorT_runs_v3:
    counter += 1
    run_start = ATs.start()
    run_end = run_start+stringlength
    print('A/T triplet (overlapping): ' +str(counter)+' extends from base ' \
    + str(run_start+1) \
    + ' to base ' +str(run_end)+' and has sequence ' +str(dna[run_start:run_end]))

print('The input sequence was ' + dna)
```

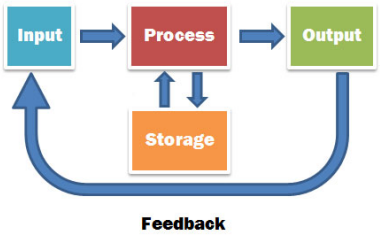
```
A/T triplet (overlapping): 1 extends from base 6 to base 8 and has sequence ATT
A/T triplet (overlapping): 2 extends from base 7 to base 9 and has sequence TTA
A/T triplet (overlapping): 3 extends from base 8 to base 10 and has sequence TAT
A/T triplet (overlapping): 4 extends from base 9 to base 11 and has sequence ATA
A/T triplet (overlapping): 5 extends from base 10 to base 12 and has sequence TAT
A/T triplet (overlapping): 6 extends from base 19 to base 21 and has sequence AAA
A/T triplet (overlapping): 7 extends from base 20 to base 22 and has sequence AAT
A/T triplet (overlapping): 8 extends from base 21 to base 23 and has sequence ATT
A/T triplet (overlapping): 9 extends from base 22 to base 24 and has sequence TTA
A/T triplet (overlapping): 10 extends from base 23 to base 25 and has sequence TAT
A/T triplet (overlapping): 11 extends from base 24 to base 26 and has sequence ATA

The input sequence was ACTGCATTATATCGTACGAAATTATACGCGCG
```

Exercises: your turn!

Make a directory within your Exercises space for today's work, and change directory to it.

```
mkdir ~/Exercises/Lecture18
cd ~/Exercises/Lecture18
```



Accession numbers

Here's a list of made-up gene accession numbers:

`xkn59438, yhdck2, eihd39d9, chdsye847, hedle3455, xjhd53e, 45da, de37dp`

Write a Python programme/script that will print only the accessions that satisfy the following criteria individually (i.e. treat each criterion separately):

- contain the number 5
- contain the letter d or e
- contain the letters d and e in that order
- contain the letters d and e in that order with a single letter between them
- contain both the letters d and e in any order
- start with x or y
- start with x or y and end with e
- contains any 3 numbers in any order
- contains 3 different numbers in the accession
- contain three or more numbers in a row
- end with d followed by either a, r or p

Give feedback
Ask for help

DNA sequence: a double digest

A file called `long_dna.txt` has been put in the following directory: `/localdisk/data/BPSM/Lecture18`

It contains a made-up DNA sequence.

- What fragment lengths will we get if we digest the sequence with a novel restriction enzyme *BpsmI*, whose recognition site is ANT*AAT, where * indicates the position of the cut site.
- What will the fragment lengths be if we do a double digest with both *BpsmI* and *BpsmII* (whose recognition site is GCRW*TG)?
- What are the sequences of the fragments themselves?

Give feedback
Ask for help

Possible answers

Accession numbers

Here's a list of made-up gene accession numbers:

`xkn59438, yhdck2, eihd39d9, chdsye847, hedle3455, xjhd53e, 45da, de37dp`

Write a Python programme/script that will print only the accessions that satisfy the following criteria individually (i.e. treat each criterion separately):

- contain the number 5
- contain the letter d or e
- contain the letters d and e in that order
- contain the letters d and e in that order with a single letter between them
- contain both the letters d and e in any order
- start with x or y
- start with x or y and end with e
- contains any 3 numbers in any order
- contains 3 different numbers in the accession
- contain three or more numbers in a row
- end with d followed by either a, r or p

The bulk of the work here will be coming up with patterns to describe the various criteria. Here's a skeleton program that will create a list to hold the accession numbers and loop over them:

```
# Import the module
import re
```

```
# INPUT list of accessions
accessions = [
  'xkn59438',
```

```
  'yhdck2',
  'eihd39d9',
  'chdsye847',
  'hedle3455',
  'xjhd53e',
  '45da',
  'de37dp']
```

```
# PROCESS list of accessions
```

```
# OUTPUT the ones we want
```

```
for acc in accessions:
    print(acc)
```

```
xkn59438
yhdck2
eihd39d9
chdsye847
hedle3455
xjhd53e
45da
de37dp
```

All we need to do is to add the criteria as we loop through each of the accessions; we can add them one by one to check each one is working first...

```
# INPUT list of accessions
accessions = [
  'xkn59438',
  'yhdck2',
  'eihd39d9',
  'chdsye847',
  'hedle3455',
  'xjhd53e',
  '45da',
  'de37dp']

outputs = []
```

```
# PROCESS list of accessions
for acc in accessions:
```

```
# contain the number 5 (could also do if '5' in acc : )
    if re.search(r'5', acc) :
        outputs.append('contain the number 5: ' + acc)
```

```
# contain the letter d or e
    if re.search(r'[de]', acc) :
        outputs.append('contain the letter d or e: ' + acc)
```

```
# contain the letters d and e (adjacent)
    if re.search(r'de', acc) :
        outputs.append('contain the letter d and e (have to be adjacent): ' + acc)
```

```
# contain the letters d and e in that order (dont have to be adjacent, but can be)
    if re.search(r'd.*e', acc) :
        outputs.append('contain the letter d and e in that order (dont have to be adjac
```

```
# contain the letters d and e in that order with a single letter between them
    if re.search(r'd.e', acc) :
        outputs.append('contain the letter d and e in that order with a single letter b
# contain both the letters d and e in any order
    if re.search(r'd', acc) and re.search(r'e', acc) :
        outputs.append('contain both the letters d and e in any order: ' + acc)
# start with x or y
# could have used if acc.startswith('x') or acc.startswith('y') :
# could have used if re.search(r'^[xy]', acc) :
    if re.search(r'(^x|^y)', acc) :
        outputs.append('start with x or y: ' + acc)
# start with x or y and end with e
# could have used if (acc.startswith('x') or acc.startswith('y')) and acc.endswith('e') :
# could have used if re.search('(^x|^y).+e$',acc) :
# could have used if re.search('"[xy].+e$',acc) :
    if re.search(r'(^x|^y)', acc) and re.search(r'(e$)', acc) :
        outputs.append('start with x or y and end with e: ' + acc)
# contains any 3 numbers in any order
    if len(re.findall(r'\d',acc)) == 3 :
        outputs.append('contains any 3 numbers in any order: ' + acc)
# contains 3 different numbers
    if len(set(re.findall(r'\d',acc))) == 3 :
        outputs.append('contains 3 different numbers: ' + acc)
# contain three or more numbers in a row
# could have used if re.search(r'[0123456789]{3,}', acc) :
    if re.search(r'\d{3,}', acc):
        outputs.append('contain three or more numbers in a row: ' + acc)
# end with d then either a, r or p
    if re.search(r'd[arp]$', acc):
        outputs.append('end with d followed by either a, r or p: ' + acc)

outputs.sort()
print(('\n').join(outputs))
```

```
contain both the letters d and e in any order: chdsye847
contain both the letters d and e in any order: de37dp
contain both the letters d and e in any order: eihd39d9
contain both the letters d and e in any order: hedle3455
contain both the letters d and e in any order: xjhd53e
contain the letter d and e (have to be adjacent): de37dp
contain the letter d and e in that order (dont have to be adjacent): chdsye847
contain the letter d and e in that order (dont have to be adjacent): de37dp
contain the letter d and e in that order (dont have to be adjacent): hedle3455
contain the letter d and e in that order (dont have to be adjacent): xjhd53e
contain the letter d and e in that order with a single letter between them: hedle3455
contain the letter d or e: 45da
contain the letter d or e: chdsye847
contain the letter d or e: de37dp
contain the letter d or e: eihd39d9
contain the letter d or e: hedle3455
contain the letter d or e: xjhd53e
contain the letter d or e: yhdck2
```

```
contain the number 5: 45da
contain the number 5: hedle3455
contain the number 5: xjhd53e
contain the number 5: xkn59438
contain three or more numbers in a row: chdsye847
contain three or more numbers in a row: hedle3455
contain three or more numbers in a row: xkn59438
contains 3 different numbers: chdsye847
contains 3 different numbers: hedle3455
contains any 3 numbers in any order: chdsye847
contains any 3 numbers in any order: eihd39d9
end with d followed by either a, r or p: 45da
end with d followed by either a, r or p: de37dp
start with x or y and end with e: xjhd53e
start with x or y: xjhd53e
start with x or y: xkn59438
start with x or y: yhdck2
```

```
# Could have used a dictionary approach to store all the different
# accessions etc in Key/value pairs
dict_approach = {}
dict_approach['contain the letter d or e'] = '45da'
```

```
# Adding another string, so we can just use +
# (and a comma to make it look nicer)
dict_approach['contain the letter d or e']=dict_approach['contain the letter d or e'] +
```

```
# What's there now?
dict_approach
{'contain the letter d or e': '45da, chdsye847'}
list(dict_approach.values())
['45da, chdsye847']
list(dict_approach.values())[0].split(',')
['45da', ' chdsye847']
list(dict_approach.items())
[('contain the letter d or e', '45da, chdsye847')]
```

etc.....

DNA sequence: a double digest

A file called `long_dna.txt` has been put in the following directory:
`/localdisk/data/BPSM/Lecture18/`

It contains a made-up DNA sequence.

- What fragment lengths will we get if we digest the sequence with a novel restriction enzyme *BpsmI*, whose recognition site is ANT*AAT, where * indicates the position of the cut site.
- What will the fragment lengths be if we do a double digest with both *BpsmI* and *BpsmII* (whose recognition site is GCRW*TG)?
- What are the sequences of the fragments themselves?
- What fragment lengths will we get if we digest the sequence with a novel restriction enzyme *BpsmI*, whose recognition site is ANT*AAT, where * indicates the position of the cut site.

Let's write a pattern for our novel enyme *BpsmI*. N means any base, so the pattern is
`A[GATC]TAAT`

We can use `re.finditer()` to find the start of all the cut sites.

```
# Open and read the remote file (or did you copy it to a new local directory?!)
dna = open('/localdisk/data/BPSM/Lecture18/long_dna.txt').read().rstrip('\n')
len(dna)
2012
```

```
# This goes WAY off the screen...!
dna
'ATGGCAATAACCCCCCGTTTCTACTTCTAGAGGAGAAAAGTATTGACATGAGCGCTCCCGGCACAAGGGCCAAAGAAGTCTCCAATTTCTTATTTCCGAATGACATGCGTCTCCTTGCGGGTAAATCACCGACCGCAATTCATAGAAGCCTGGGGGAACAGATAGGTCTAATTAGCTTAAGAGAGTAAATCCTGGGATCATTCAGTAGTAACCATAAACTTACGCTGGGGCTTCTTC
```

```
# Cool way to make the long string look nicer printed on the screen...
print("\n".join(re.findall('.{1,60}', dna)))
```

```
ATGGCAATAACCCCCCGTTTCTACTTCTAGAGGAGAAAAGTATTGACATGAGCGCTCCCG
GCACAAGGGCCAAAGAAGTCTCCAATTTCTTATTTCCGAATGACATGCGTCTCCTTGCGG
GTAAATCACCGACCGCAATTCATAGAAGCCTGGGGGAACAGATAGGTCTAATTAGCTTAA
GAGAGTAAATCCTGGGATCATTCAGTAGTAACCATAAACTTACGCTGGGGCTTCTTCGGC
GGATTTTTACAGTTACCAACCAGGAGATTTGAAGTAAATCAGTTGAGGATTTAGCCGCGC
TATCCGGTAATCTCCAAATTAAAACATACCGTTCCATGAAGGCTAGAATTACTTACCGGC
CTTTTCCATGCCTGCGCTATACCCCCCCACTCTCCCGCTTATCCGTCCGAGCGGAGGCAG
TGCGATCCTCCGTTAAGATATTCTTACGTGTGACGTAGCTATGTATTTTGCAGAGCTGGC
GAACGCGTTGAACACTTCACAGATGGTAGGGATTCGGGTAAAGGGCGTATAATTGGGGAC
TAACATAGGCGTAGACTACGATGGCGCCAACTCAATCGCAGCTCGAGCGCCCTGAATAAC
GTACTCATCTCAACTCATTCTCGGCAATCTACCGAGCGACTCGATTATCAACGGCTGTCT
AGCAGTTCTAATCTTTTGCCAGCATCGTAATAGCCTCCAAGAGATTGATGATAGCTATCG
GCACAGAACTGAGACGGCGCCGATGGATAGCGGACTTTCGGTCAACCACAATTCCCCACG
GGACAGGTCCTGCGGTGCGCATCACTCTGAATGTACAAGCAACCCAAGTGGGCCGAGCCT
GGACTCAGCTGGTTCCTGCGTGAGCTCGAGACTCGGGATGACAGCTCTTTAAACATAGAG
CGGGGGCGTCGAACGGTCGAGAAAGTCATAGTACCTCGGGTACCAACTTACTCAGGTTAT
TGCTTGAAGCTGTACTATTTTAGGGGGGGAGCGCTGAAGGTCTCTTCTTCTCATGACTGA
ACTCGCGAGGGTCGTGAAGTCGGTTCCTTCAATGGTTAAAAAACAAAGGCTTACTGTGCG
CAGAGGAACGCCCATCTAGCGGCTGGCGTCTTGAATGCTCGGTCCCCTTTGTCATTCCGG
ATTAATCCATTTCCCTCATTCACGAGCTTGCGAAGTCTACATTGGTATATGAATGCGACC
TAGAAGAGGGCGCTTAAAATTGGCAGTGGTTGATGCTCTAAACTCCATTTGGTTTACTCG
TGCATCACCGCGATAGGCTGACAAAGGTTTAACATTGAATAGCAAGGCACTTCCGGTCTC
AATGAACGGCCGGGAAAGGTACGCGCGCGGTATGGGAGGATCAAGGGGCCAATAGAGAGG
CTCCTCTCTCACTCGCTAGGAGGCAAATGTAAAACAATGGTTACTGCATCGATACATAAA
ACATGTCCATCGGTTGCCCAAAGTGTTAAGTGTCTATCACCCCTAGGGCCGTTTCCCGCA
TATAAACGCCAGGTTGTATCCGCATTTGATGCTACCGTGGATGAGTCTGCGTCGAGCGCG
CCGCACGAATGTTGCAATGTATTGCATGAGTAGGGTTGACTAAGAGCCGTTAGATGCGTC
GCTGTACTAATAGTTGTCGACAGACCGTCGAGATTAGAAAATGGTACCAGCATTTTCGGA
GGTTCTCTAACTAGTATGGATTGCGGTGTCTTCACTGTGCTGCGGCTACCCATCGCCTGA
AATCCAGCTGGTGTCAAGCCATCCCCTCTCCGGGACGCCGCATGTAGTGAAACATATACG
TTGCACGGGTTCACCGCGGTCCGTTCTGAGTCGACCAAGGACACAATCGAGCTCCGATCC
GTACCCTCGACAAACTTGTACCCGACCCCCGGAGCTTGCCAGCTCCTCGGGTATCATGGA
GCCTGTGGTTCATCGCGTCCGATATCAAACTTCGTCATGATAAAGTCCCCCCCTCGGGAG
TACCAGAGAAGATGACTACTGAGTTGTGCGAT
```

```
BpsmI='A[GATC]TAAT'
print('BpsmI cuts at:',BpsmI)
```

```
# Find the sites, incrementing by three to account for where the enzyme cuts in the recognition sequence
```

```
for matching in re.finditer(BpsmI, dna):
    print(matching.start()+3)
```

```
BpsmI cuts at: A[GATC]TAAT
1143
1628
```

Once we've got the cut positions, we calculate the distance from the current cut site to the previous one (or to the start of the sequence).

```
# Start: open and read the file
```

```
dna = open('/localdisk/data/BPSM/Lecture18/long_dna.txt').read().rstrip('\n')
last_cut = 0
findnum=0
for matching in re.finditer(BpsmI, dna):
    findnum += 1
    cut_position = matching.start() + 3
# Distance from the current cut site to the previous one
    fragment_size = cut_position - last_cut
    print('Fragment size is ' + str(fragment_size))
    last_cut = cut_position
# We also have to remember the last fragment, from the last cut to the end:
    if findnum == len(list(re.finditer(BpsmI, dna))) :
       fragment_size = len(dna) - last_cut
       print('Fragment size is ' + str(fragment_size))
```

```
Fragment size is 1143
Fragment size is 485
Fragment size is 384
```

- What will the fragment lengths be if we do a double digest with both *BpsmI* and *BpsmII* (whose recognition site is GCRW*TG)?

Doing the same for two enzymes is trickier. Ambiguity code 'R' corresponds to the purines A or G, while 'W' corresponds to A or T, so we will need to 'convert' it too...

```
# First, define both enzymes sites
BpsmI='A[GATC]TAAT'
BpsmII='GC[AG][AT]TG'
```

```
# Make a list to store the cut positions for both enzymes
all_cuts = []
# Add cut positions for BpsmI
for match in re.finditer(BpsmI, dna):
    all_cuts.append(match.start() + 3)
# Add cut positions for BpsmII
for match in re.finditer(BpsmII, dna):
    all_cuts.append(match.start() + 4)

print(all_cuts)
```

```
[1143, 1628, 488, 1577]
```

```
# These aren't in sequential order, so just sort them
all_cuts.sort()
all_cuts
```

```
[488, 1143, 1577, 1628]
```

Now we can go through the list of all cuts with the same logic as before.

```
# Double digest run
last_cut = 0
counter = 0
for cut_position in all_cuts:
    counter +=1
    fragment_size = cut_position - last_cut
    print('Fragment '+str(counter)+' size is ' + \
       str(fragment_size) +': '+ str(last_cut)+ ' to ' +str(cut_position) )
    last_cut = cut_position
# Now the last fragment
fragment_size = len(dna) - last_cut
counter +=1
```

```
print('Fragment '+str(counter)+' size is ' + \
  str(fragment_size) +': '+ str(last_cut)+ ' to ' +str(len(dna)) )
```

```
Fragment 1 size is 488: 0 to 488
Fragment 2 size is 655: 488 to 1143
Fragment 3 size is 434: 1143 to 1577
Fragment 4 size is 51: 1577 to 1628
Fragment 5 size is 384: 1628 to 2012
```

- What are the sequences of the fragments themselves?

```
# We have just worked out where the enzymes cut the DNA sequence,
# so all we need to do is to use the cut positions as index positions to
# substring the dna sequence string!
```

```
# Let's use a dictionary to store the fragment sequences
fragment_sequences = {}
```

```
# Double digest run
last_cut = 0
counter = 0
for cut_position in all_cuts:
    counter +=1
    fragment_size = cut_position - last_cut
    print('Fragment '+str(counter)+' size is ' + \
       str(fragment_size) +': '+ str(last_cut)+ ' to ' +str(cut_position) )
# Get the sequence substring
    fragment_sequences['Fragment'+str(counter)] = dna[last_cut:cut_position]
    print(fragment_sequences['Fragment'+str(counter)])
# Get the fragment start and end
    fragends = dna[last_cut:cut_position][0:6] + '...' + dna[last_cut:cut_position][-6:]
    print('Fragment '+str(counter)+ ' has ends: '+fragends+'\n')
    last_cut = cut_position
# Now the last fragment
fragment_size = len(dna) - last_cut
counter +=1
print('Fragment '+str(counter)+' size is ' + \
  str(fragment_size) +': '+ str(last_cut)+ ' to ' +str(len(dna)) )
fragment_sequences['Fragment'+str(counter)] = dna[last_cut:]
print(fragment_sequences['Fragment'+str(counter)])
fragends = dna[last_cut:][0:6] + '...' + dna[last_cut:][-6:]
print('Fragment has ends: '+fragends)
```

```
Fragment 1 size is 488: 0 to 488
ATGGCAATAACCCCCCGTTTCTACTTCTAGAGGAGAAAAGTATTGACATGAGCGCTCCCGGCACAAGGGCCAAAGAAGTCTCC
Fragment 1 has ends: ATGGCA...ACGCGT

Fragment 2 size is 655: 488 to 1143
TGAACACTTCACAGATGGTAGGGATTCGGGTAAAGGGCGTATAATTGGGGACTAACATAGGCGTAGACTACGATGGCGCCAAC
Fragment 2 has ends: TGAACA...CGGATT

Fragment 3 size is 434: 1143 to 1577
AATCCATTTCCCTCATTCACGAGCTTGCGAAGTCTACATTGGTATATGAATGCGACCTAGAAGAGGGCGCTTAAAATTGGCAC
```

```
Fragment 3 has ends: AATCCA...TTGCAA

Fragment 4 size is 51: 1577 to 1628
TGTATTGCATGAGTAGGGTTGACTAAGAGCCGTTAGATGCGTCGCTGTACT
Fragment 4 has ends: TGTATT...TGTACT

Fragment 5 size is 384: 1628 to 2012
AATAGTTGTCGACAGACCGTCGAGATTAGAAAATGGTACCAGCATTTTCGGAGGTTCTCTAACTAGTATGGATTGCGGTGTCT
Fragment has ends: AATAGT...TGCGAT
```

```
# Show all the sequences
```

```
print(('\n########\n').join(list(fragment_sequences.values())))
```

```
TGAACACTTCACAGATGGTAGGGATTCGGGTAAAGGGCGTATAATTGGGGACTAACATAGGCGTAGACTACGATGGCGCCAAC
########
TGTATTGCATGAGTAGGGTTGACTAAGAGCCGTTAGATGCGTCGCTGTACT
########
AATCCATTTCCCTCATTCACGAGCTTGCGAAGTCTACATTGGTATATGAATGCGACCTAGAAGAGGGCGCTTAAAATTGGCAC
########
ATGGCAATAACCCCCCGTTTCTACTTCTAGAGGAGAAAAGTATTGACATGAGCGCTCCCGGCACAAGGGCCAAAGAAGTCTCC
########
AATAGTTGTCGACAGACCGTCGAGATTAGAAAATGGTACCAGCATTTTCGGAGGTTCTCTAACTAGTATGGATTGCGGTGTCT
```

```
# Are the fragments actually adjacent? Quick check!
```

```
# End of Fragment 1 ACGCGT should be next to
# beginning of Fragment 2 TGAACA
# so lets use ACGCGTTGAACA as a query against our sequence
```

```
re.search(r'ACGCGTTGAACA',dna)
```

```
<_sre.SRE_Match object; span=(482, 494), match='ACGCGTTGAACA'>
```

Yes, that's consistent, so job is (probably) done!
Could/should probably make the code into a function....?

---

git and GitHub time!

```
# Local repository actions (the absolute minimum set of commands required here...)
```

```
git lo
```

```
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz
```

```
        modified:   ../Lecture06/run_this_script_v1

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details
        ../Lecture07/Cosmoscarta.nuc.fa
        ../Lecture07/Cosmoscarta.nuc.gis
        ../Lecture07/myfileofaccessions.txt
        ../Lecture12/AJ223353.fasta
        ../Lecture12/AJ223353.genbank
        ../Lecture12/AJ223353_noheader.fasta
        ../Lecture12/AJ223353_noheader.fasta2
        ../Lecture12/AJ223353_noheader.fasta3
        ../Lecture12/All_exons.fasta
        ../Lecture12/All_noncodings.fasta
        ../Lecture12/local_exon01.fasta
        ../Lecture12/local_exon02.fasta
        ../Lecture12/local_intron01.fasta
        ../Lecture12/my_dna.txt
        ../Lecture12/plain_genomic_seq.txt
        ../Lecture12/quick_blast_check.out
        ../Lecture12/remote_exon01.fasta
        ../Lecture12/remote_noncoding01.fasta
        ../Lecture12/remote_noncoding02.fasta
        ../Lecture12/shutilversion.txt
        ../Lecture13/100_199/
        ../Lecture13/200_299/
        ../Lecture13/300_399/
        ../Lecture13/400_499/
        ../Lecture13/500_599/
        ../Lecture13/600_699/
        ../Lecture13/700_799/
        ../Lecture13/800_899/
        ../Lecture13/900_999/
        ../Lecture13/Cleaned_sequences.txt
        ../Lecture13/Cleaned_sequences2.txt
        ../Lecture13/Cleaned_sequences3.txt
        ../Lecture13/Exercise2_coding_seqence.fasta
        ../Lecture13/Lecture13.tar.gz
        ../Lecture13/dna_files/
        ../Lecture13/exons.txt
        ../Lecture13/genomic_dna2.txt
        ../Lecture13/input.txt
        ../Lecture13/remote_exon01.fasta
        ../Lecture14/__pycache__/
        ../Lecture14/data.csv
        ../Lecture16/__pycache__/
        ../Lecture17/All_the_Arabidopsis_ones.tsv
        ../Lecture17/eukaryotes.txt
        ../Lecture17/playtime/
        ./

no changes added to commit (use "git add" and/or "git commit -a")
```

You might want to add other files?

ls -1

```
long_dna.txt
regex_pies.py
```

git add long_dna.txt *.py

git commit -m "Scripts from the python regex lecture"

```
[master 41e1ac8] Scripts from the python regex lecture
 2 files changed, 226 insertions(+)
 create mode 100755 Lecture18/long_dna.txt
 create mode 100644 Lecture18/regex_pies.py
```

Push our local repository to GitHub

git remote

```
AllLectures
Lecture07
origin
```

git push AllLectures master

```
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 256 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 3.35 KiB | 1.68 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/B123456-2121/BPSM_LectureStuff.git
   097800e..41e1ac8  master -> master
```

git lo

```
41e1ac8 (HEAD -> master, AllLectures/master) Scripts from the python regex lecture
097800e Script from the python pandas lecture
1d2319e Scripts from the python dictionaries lecture
c49df45 Scripts from the python functions lecture
3b8bf41 My scripts from the python conditionals exercises Py04
8f58f4c Scripts from the python lists and loops lecture 3
e444ef8 Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
fbe5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!

Sources used